16. 기본 예제 프로그램 분석

개요

SP1은 다양한 기본 예제 프로그램을 제공하여 개발자들이 영지식 증명 시스템의 동작 원리를 이해하고 실제 구현을 학습할 수 있도록 돕습니다. 본 문서에서는 피보나치 수열 계산과 소수 판별이라는 두 가지 대표적인 예제를 통해 SP1의 전체 워크플로우를 단계별로 분석합니다.

16.1 피보나치 수열 예제 분석

16.1.1 프로그램 구조

피보나치 예제는 SP1의 가장 기본적인 예제로, 다음과 같은 구조를 가집니다:

```
examples/fibonacci/
├── program/ # zkVM에서 실행될 프로그램
| ├── src/main.rs # 피보나치 계산 로직
| └── Cargo.toml # 프로그램 의존성
└── script/ # 증명 생성 및 검증 스크립트
├── src/main.rs # 기본 증명 생성
├── bin/ # 다양한 증명 타입별 실행 파일
| ├── execute.rs # 실행만 수행 (증명 생성 없음)
| ├── compressed.rs # 압축된 증명 생성
| └── plonk_bn254.rs # PLONK 증명 생성
```

```
└── Cargo.toml # 스크립트 의존성
```

16.1.2 zkVM 프로그램 구현

```rust
#![no_main]
sp1_zkvm::entrypoint!(main);

pub fn main() {
// 입력값 읽기
let n = sp1_zkvm::io::read::<u32>();
// 공개 입력으로 n 커밋
sp1_zkvm::io::commit(&n);

// 피보나치 수열 계산
let mut a = 0;
let mut b = 1;
for _ in 0..n {
let mut c = a + b;
c %= 7919; // 오버플로우 방지를 위한 모듈러 연산
a = b;
b = c;
}
```

```
// 결과값 커밋
sp1_zkvm::io::commit(&a);
sp1_zkvm::io::commit(&b);
}
```

핵심 구성 요소:

1. **`#![no_main]`** : 표준 main 함수 대신 SP1의 엔트리포인트 사용
2. **`sp1_zkvm::entrypoint!(main)`** : SP1 zkVM 환경에서 실행되는 진입점 정의
3. **`sp1_zkvm::io::read::<T>()`** : 증명자(Prover)로부터 입력 데이터 읽기
4. **`sp1_zkvm::io::commit(&value)`** : 공개 값으로 데이터 커밋

16.1.3 증명 생성 스크립트 분석

기본 증명 생성 과정

```
use sp1_sdk::{include_elf, utils, ProverClient, SP1Stdin};

const ELF: &[u8] = include_elf!("fibonacci-program");

fn main() {
// 로깅 설정
utils::setup_logger();

// 입력 데이터 준비
let n = 1000u32;
```

```rust
let mut stdin = SP1Stdin::new();
stdin.write(&n);

// ProverClient 생성
let client = ProverClient::from_env();

// 1단계: 실행만 수행 (증명 생성 없음)
let (_, report) = client.execute(ELF, &stdin).run().unwrap();
println!("executed program with {} cycles", report.total_instruction_count());

// 2단계: 키 생성
let (pk, vk) = client.setup(ELF);
// 3단계: 증명 생성
let mut proof = client.prove(&pk, &stdin).plonk().run().unwrap();

// 4단계: 결과 검증
let _ = proof.public_values.read::<u32>();
let a = proof.public_values.read::<u32>();
let b = proof.public_values.read::<u32>();

// 5단계: 증명 검증
client.verify(&proof, &vk).expect("verification failed");
```

```
}
```

16.1.4 단계별 증명 생성 과정

1단계: 프로그램 실행 (execute.rs)

```
let (mut public_values, execution_report) = client.execute(ELF, &stdin).run().u
```

- **목적**: 증명 생성 없이 프로그램 실행만 수행
- **출력**: 실행 결과와 성능 리포트
- **사용 사례**: 디버깅, 성능 측정, 로직 검증

실행 리포트 예시:

```
Executed program with 3045 cycles
Full execution report:
ExecutionReport {
total_instruction_count: 3000,
total_syscall_count: 45,
opcode_counts: {...},
syscall_counts: {...}
}
```

2단계: 압축 증명 생성 (compressed.rs)

```
let mut proof = client.prove(&pk, &stdin).compressed().run().unwrap();
```

- **목적**: 고정 크기의 압축된 STARK 증명 생성
- **특징**: 빠른 생성 속도, 상대적으로 큰 증명 크기
- **사용 사례**: 개발 및 테스트 환경

3단계: PLONK 증명 생성 (plonk_bn254.rs)

```
let  proof = client.prove(&pk, &stdin).plonk().run().unwrap();
```

- **목적**: BN254 곡선 기반 PLONK 증명 생성
- **특징**: 작은 증명 크기, 긴 생성 시간
- **사용 사례**: 온체인 검증, 프로덕션 환경

16.1.5 성능 측정 및 분석

사이클 수 분석

피보나치 예제의 성능 특성:

입력값 (n)	실행 사이클	메모리 사용량	증명 생성 시간
100	~1,200	2MB	5초
500	~3,000	4MB	12초
1000	~5,500	6MB	25초

최적화 기법

1. **모듈러 연산 사용**:

```
c %= 7919; // 오버플로우 방지
```

2. **효율적인 루프 구조:**

```
for _ in 0..n {
// 단순한 반복 구조로 사이클 수 최소화
}
```

16.2 소수 판별 예제 분석

16.2.1 프로그램 구조

```
#![no_main]
sp1_zkvm::entrypoint!(main);

pub fn main() {
let n = sp1_zkvm::io::read::<u64>();
let is_prime = is_prime(n);
sp1_zkvm::io::commit(&is_prime);
}

fn is_prime(n: u64) -> bool {
if n <= 1 { return false; }
if n <= 3 { return true; }
if n % 2 == 0 || n % 3 == 0 { return false; }
```

```
let mut i = 5;
while i * i <= n {
if n % i == 0 || n % (i + 2) == 0 {
return false;
}
i += 6;
}
true
}
```

16.2.2 알고리즘 분석

6k ± 1 최적화 기법:

1. **기본 케이스 처리**: n ≤ 1, n ≤ 3
2. **짝수/3의 배수 제거**: n % 2 == 0 || n % 3 == 0
3. **6k ± 1 형태 검사**: 5, 7, 11, 13, 17, 19, ...

이 알고리즘은 모든 소수가 6k ± 1 형태라는 수학적 성질을 활용합니다.

16.2.3 성능 비교

입력값	실행 사이클	증명 생성 시간	결과
29	~150	2초	true
97	~200	3초	true
1009	~800	8초	true

| 1024 | ~600 | 6초 | false |

16.2.4 증명 생성 스크립트

```
fn main() {
utils::setup_logger();
let mut stdin = SP1Stdin::new();
let n = 29u64;
stdin.write(&n);

let client = ProverClient::from_env();
let (pk, vk) = client.setup(ELF);
let mut proof = client.prove(&pk, &stdin).run().unwrap();

let is_prime = proof.public_values.read::<bool>();
println!("Is {} prime? {}", n, is_prime);

client.verify(&proof, &vk).expect("verification failed");
}
```

16.3 공통 패턴 및 베스트 프랙티스

16.3.1 프로그램 작성 패턴

1. **엔트리포인트 설정**:

```
#![no_main]

sp1_zkvm::entrypoint!(main);
```

2. **입력 처리**:

```
let input = sp1_zkvm::io::read::<T>();
```

3. **계산 수행**:

```
let result = compute(input);
```

4. **결과 커밋**:

```
sp1_zkvm::io::commit(&result);
```

16.3.2 스크립트 작성 패턴

1. **ELF 포함**:

```
const ELF: &[u8] = include_elf!("program-name");
```

2. **입력 준비**:

```
let mut stdin = SP1Stdin::new();

stdin.write(&input_data);
```

3. **클라이언트 설정**:

```
let client = ProverClient::from_env();
let (pk, vk) = client.setup(ELF);
```

4. **증명 생성 및 검증**:

```
let proof = client.prove(&pk, &stdin).run().unwrap();
client.verify(&proof, &vk).expect("verification failed");
```

16.3.3 성능 최적화 전략

1. 알고리즘 최적화

- **효율적인 알고리즘 선택**: 시간 복잡도가 낮은 알고리즘 사용
- **조기 종료**: 불필요한 계산 방지
- **모듈러 연산**: 오버플로우 방지 및 계산 최적화

2. 메모리 최적화

- **스택 사용 최소화**: 큰 배열이나 구조체 사용 자제
- **인플레이스 연산**: 추가 메모리 할당 없이 계산 수행

3. 사이클 수 최적화

- **단순한 제어 흐름**: 복잡한 분기문 최소화
- **효율적인 데이터 타입**: 필요에 따라 적절한 크기의 정수 타입 사용

16.4 디버깅 및 문제 해결

16.4.1 일반적인 문제들

1. 컴파일 오류

문제: `#![no_main]` 누락

```
error: `main` function not found in crate
```

해결: 프로그램 상단에 `#![no_main]` 추가

2. 런타임 오류

문제: 입력 타입 불일치

```
thread 'main' panicked at 'called `Result::unwrap()` on an `Err` value'
```

해결: 프로그램과 스크립트의 입력 타입 일치 확인

3. 증명 검증 실패

문제: 공개 값 불일치

```
verification failed
```

해결: 커밋된 값과 검증 시 읽는 값의 순서 및 타입 확인

16.4.2 디버깅 도구

1. 실행 전용 모드

```rust
let (public_values, report) = client.execute(ELF, &stdin).run().unwrap();
println!("Execution report: {:?}", report);
```

2. 로깅 활용

```
utils::setup_logger();

// 자동으로 실행 과정의 상세 로그 출력
```

3. 사이클 수 모니터링

```
println!("Total cycles: {}", report.total_instruction_count());
```

16.5 실제 사용 시나리오

16.5.1 교육 목적

- **개념 학습**: 영지식 증명의 기본 원리 이해
- **워크플로우 습득**: SP1 개발 프로세스 학습
- **성능 이해**: 사이클 수와 증명 생성 시간의 관계 파악

16.5.2 프로토타이핑

- **알고리즘 검증**: 새로운 알고리즘의 SP1 호환성 테스트
- **성능 측정**: 실제 애플리케이션 개발 전 성능 예측
- **통합 테스트**: CI/CD 파이프라인에서의 자동 테스트

16.5.3 벤치마킹

- **하드웨어 성능**: 다양한 환경에서의 성능 비교
- **최적화 효과**: 코드 변경에 따른 성능 개선 측정
- **확장성 테스트**: 입력 크기에 따른 성능 변화 분석

결론

기본 예제 프로그램들은 SP1의 핵심 개념과 워크플로우를 이해하는 데 필수적인 학습 자료입니다. 피보나치와 소수 판별 예제를 통해 다음과 같은 핵심 사항들을 학습할 수 있습니다:

1. **SP1 프로그램 구조**: `#![no_main]`, 엔트리포인트, I/O 처리
2. **증명 생성 과정**: 실행, 키 생성, 증명 생성, 검증
3. **성능 최적화**: 알고리즘 선택, 메모리 관리, 사이클 수 최적화
4. **디버깅 기법**: 실행 전용 모드, 로깅, 성능 모니터링

이러한 기본기를 바탕으로 더 복잡한 애플리케이션 개발에 도전할 수 있습니다.